

```

import streamlit as st
import altair as alt
import plotly.express as px
import pandas as pd
import numpy as np
from datetime import datetime
import joblib
from track_utils import create_page_visited_table, add_page_visited_details,
view_all_page_visited_details, add_prediction_details,
view_all_prediction_details, create_emotionclf_table, IST
pipe_lr = joblib.load(open("./models/emotion_classifier_pipe_lr.pkl", "rb"))

def predict_emotions(docx):
    results = pipe_lr.predict([docx])
    return results[0]

def get_prediction_proba(docx):
    results = pipe_lr.predict_proba([docx])
    return results

emotions_emoji_dict = {"anger", "disgust", "fear", "happy", "joy", "neutral",
"sad", "sadness", "shame", "surprise"}

def main():
    st.title("Emotion Classifier App")
    menu = ["Home", "Monitor", "About"]
    choice = st.sidebar.selectbox("Menu", menu)
    create_page_visited_table()
    create_emotionclf_table()
    if choice == "Home":
        add_page_visited_details("Home", datetime.now(IST))
        st.subheader("Emotion Detection in Text")

        with st.form(key='emotion_clf_form'):
            raw_text = st.text_area("Type Here")
            submit_text = st.form_submit_button(label='Submit')

        if submit_text:
            col1, col2 = st.columns(2)

            prediction = predict_emotions(raw_text)
            probability = get_prediction_proba(raw_text)

            add_prediction_details(raw_text, prediction, np.max(probability),
datetime.now(IST))

            with col1:
                st.success("Original Text")

```

```

        st.write(raw_text)

        st.success("Prediction")
        emoji_icon = emotions_emoji_dict[prediction]
        st.write("{}:{}".format(prediction, emoji_icon))
        st.write("Confidence:{}".format(np.max(probability)))

    with col2:
        st.success("Prediction Probability")
        proba_df = pd.DataFrame(probability, columns=pipe_lr.classes_)
        proba_df_clean = proba_df.T.reset_index()
        proba_df_clean.columns = ["emotions", "probability"]

        fig =
alt.Chart(proba_df_clean).mark_bar().encode(x='emotions', y='probability',
color='emotions')
        st.altair_chart(fig, use_container_width=True)

    elif choice == "Monitor":
        add_page_visited_details("Monitor", datetime.now(IST))
        st.subheader("Monitor App")

        with st.expander("Page Metrics"):
            page_visited_details =
pd.DataFrame(view_all_page_visited_details(), columns=['Page Name', 'Time of
Visit'])
            st.dataframe(page_visited_details)

            pg_count = page_visited_details['Page
Name'].value_counts().rename_axis('Page Name').reset_index(name='Counts')
            c = alt.Chart(pg_count).mark_bar().encode(x='Page Name',
y='Counts', color='Page Name')
            st.altair_chart(c, use_container_width=True)

            p = px.pie(pg_count, values='Counts', names='Page Name')
            st.plotly_chart(p, use_container_width=True)

        with st.expander('Emotion Classifier Metrics'):
            df_emotions = pd.DataFrame(view_all_prediction_details(),
columns=['Rawtext', 'Prediction', 'Probability', 'Time_of_Visit'])
            st.dataframe(df_emotions)

            prediction_count =
df_emotions['Prediction'].value_counts().rename_axis('Prediction').reset_index
(name='Counts')
            pc = alt.Chart(prediction_count).mark_bar().encode(x='Prediction',
y='Counts', color='Prediction')
            st.altair_chart(pc, use_container_width=True)

```

```
    else:
        add_page_visited_details("About", datetime.now(IST))

if __name__ == '__main__':
    main()
```